This article has been accepted for publication in IEEE Embedded Systems Letters. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/LES.2026.3677498

MQIL: Model Checking based Quantification of Information Leakage

Priyanka Panigrahi* and Chandan Karfa†
* National Institute of Technology Tiruchirappalli, India † Indian Institute of Technology Guwahati, India
*ppriyanka@nitt.edu, †ckarfa@iitg.ac.in

***Abstract*—Compiler optimization may introduce information leakage (IL) in a program. This opens up the scope of side-channel attacks through which secret inputs, such as keys, may leak via intermediate variables in cryptographic applications. Taint analysis is a well-known approach that tracks the information flow in a program. However, it has the problem of either under-tainting, leading to false negatives, or over-tainting, leading to false positive scenarios. In this work, we overcome these problems by tracking the information flow in a C program using a bounded model checker. Conceptually, we create a miter program of the input program and add assertions to verify the security property related to information flow using a model checker. In our experiment, we run the proposed approach for various cryptographic applications and show that it can successfully track the IL. Our experiments reveal that LLVM actually introduces IL during optimizations.**

***Index Terms*—Security, information flow, model checking, CBMC**

I. Introduction and Related Work

The functional correctness of a compiler is well studied in the literature [1]. However, a compiler optimization (Opt) may be correct yet insecure [2]. Many well-known compiler optimizations like dead store elimination [3], static single assignment, code motion, etc., are shown to be not secure [4]. Thus, it is a threat to cryptographic programs running in safety-critical systems, cyber-physical systems, and secure embedded systems. The sensitive inputs, like keys, may be leaked through side channel attacks due to ILs introduced by the optimizations phase of a compiler [5], [6].

The security verification of a compiler optimization is an undecidable problem [7]. The primary approach for identifying information flow is taint analysis, which tracks how sensitive information flows through the program and if it is leaked to public observers. There have been a few recent taint analysis-based attempts [8] to quantify the IL for security verification of compiler optimizations with respect to information flow. However, taint analysis-based methods suffer from well-known over-tainting or under-tainting problems [8]. Therefore, we need a scalable and practical method to quantify the information leakage in a program.

We present a framework for checking security preservation under compiler optimizations and demonstrate its usefulness through a full instantiation for C, a language whose low-level semantics pose fundamental challenges for such reasoning. We measure the overall IL in a program with respect to information flow using a model checker. Specifically, we have reduced the quantification of IL problem in a program into a set of property checking problems on a mitered version of the program (as shown in Fig. 1). Our proposed method measures the IL precisely for programs with bounded loops. To demonstrate its practical use for C language, we have shown how to model the miter program

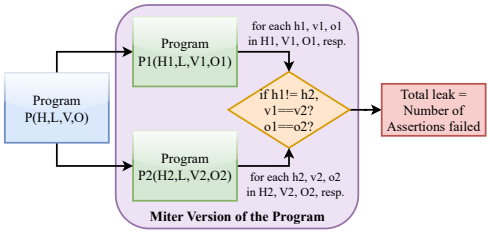


Fig. 1: Our overall flow of quantification method

for all C constructs. As shown in Sec. III, such construction is not trivial. To the best of our knowledge, this is the first work that successfully quantifies the IL in any C program using Bounded Model Checker for C (CBMC) and is scalable enough to handle large cryptographic benchmarks. The method is also extended to check the relative security (RS) between a source program and its optimized version. Experiments on security benchmarks show our approach effectively measures IL, revealing that LLVM introduces vulnerabilities during optimization in most cryptographic benchmarks.

Related Works: Authors in [4] identified the correctness-security gap due to various compiler optimizations. Most of the existing methods analyze the security properties of a specific optimization. Authors in [2] formally defined the information flow preservation in a program transformation. They considered observation points to memory at some fixed points [2]. A leaky triple has been defined as a security property to identify and restore the source-level security of dead store elimination of a program. However, none of the existing methods can measure the overall IL in a program. Authors in [8] proposed a quantification method for IL in a program. The authors in [9] proposed a translation validation method to verify RS between two programs. However, their Finite State Machine with Datapaths (FSMD) based approach cannot handle arrays, functions, structures, and many other constructs of a program. Thus, the method in [8], [9] is not applicable to practical cryptographic benchmarks. This work overcomes these limitations.

II. Motivation

The information leakage in a program is due to explicit and implicit information flow from high or sensitive inputs (H_in) to program variables. The P_1 in Fig. 2(b) is generated after applying copy propagation and code motion to the P_0 in Fig. 2(a). P_1 is functionally equivalent to P_0. Assume h is a sensitive input, i.e., $h \in H_in$. Clearly, there is an explicit leak of h due to the direct information flow from h to array u at Line 6 of P_0.

Authorized licensed use limited to: Indian Institute of Technology (ISM) Dhanbad. Downloaded on August 04,2026 at 12:14:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted,
but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Embedded Systems Letters. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/LES.2026.3677498

There is no leak of h through variable b and c in the source program, as it only depends on the low or non-sensitive input (L_in) x and y. However, c is leaking h *implicitly* due to the influence of $h \in H_in$ in the condition (called culprit block) of the optimized program P_1 and due to different symbolic values across the parallel paths of the conditional block. Conventional taint analysis fails to identify such implicit leaks, a limitation commonly referred to as the under-tainting problem. To address this issue, taint analysis may mark all assignments inside a culprit as tainted. This causes over-tainting since there is no leak of h through b in P_1, due to the same symbolic values irrespective of the path it follows. In Fig. 2(a), there is an explicit information flow from h to $u[1]$ in the first iteration of the loop. Now, in the second iteration, $v[2]$ is leaking h. Eventually, h is propagating to all $v[i], i = 1, \ldots, n$; the identification of this fact is called the fixed point of loop leak. Thus, we need a precise analysis of implicit flows as leakage and loop, to measure the overall leak in the program.

```
1  void func(x,y,z,n,h)
2  { i = 1; u[0] = 0;
3    b = x - y; c = x + y;
4    while(i < n)
5    { v[i] = u[i-1] + i;
6      u[i] = h; i++; }
7    if(u[i-1] > 0){
8      a = b - c;}
9    else{ a = b + z;}
10   out = v[i-1] + a; }
```

(a) Source program P_0

```
1  void func(x,y,z,n,h)
2  { i = 1; v[1] = 1;
3    while(i+1 < n)
4    { v[i] = h + i; i++; }
5    if(h > 0){
6      b = x - y; c = x + y;
7      a = b - c;}
8    else{ b = x - y;
9      a = b + z;}
10   out = v[i-1] + a; }
```

(b) Optimized program P_1

Fig. 2: Under-tainting and Over-tainting problem

The motivational question here is *can we model the information flow property of a program as user-specified assertions and utilize the power of CBMC to verify them?* In this work, our objective is to model these explicit flows, implicit flows, and fixed points of loops as user-specified assertions to be verified by CBMC.

III. Our Quantification Model

Let $P(H, L, V, O)$ be a program, where H and L are the sets of high/sensitive and low/insensitive inputs, respectively, V and O are the sets of program variables and output variables, respectively. The program P consists of assignment statements, control blocks, loops, functions, etc. We create two copies of P at the abstract level as $P1(H1, L, V1, O1)$ and $P2(H2, L, V2, O2)$. Each element v in $H \cup V \cup O$ are copied as $v1$ and $v2$ in $P1$ and $P2$, respectively. It may be noted that we do not create copies of the low input set L. Each statement of the program is rewritten with $(H1, L, V1, O1)$ variables in $P1$ and with $(H2, L, V2, O2)$ variables in $P2$. To quantify the IL, we add assertions of the form $v1 == v2, \forall v1 \in V1 \cup O1$ and $\forall v2 \in V2 \cup O2$. We assume that $h1 \neq h2$, $\forall h1 \in H1$ and $\forall h2 \in H2$, where $h1$ and $h2$ are corresponding to the sensitive input $h \in H$ in the program P. *For different values of the h, if the assertion fails for a program or output variable v, it implies that there is an information flow from h to v in the original program P.* The combined code $P1$, $P2$, and all assumptions and assertions are given as input to the model checking tool. *The total leak is defined by the number of asserts failed in this model checking.* The abstract idea of the proposed method for quantification of IL is presented in Fig. 1. Our quantification model is generic and can be adopted for any language.

IV. Quantification of Information Leakage in C

Although the core idea of our MQIL is to create two copies of a program and then check the assertion for each program variable and output with the assumption that sensitive input contains a different value, there are several language-specific challenges to be addressed in its language-specific implementation. In this Section, we discuss how MQIL can be implemented for a C program for which we use CBMC as the model checker.

```
1  #include<assert.h>
2  main(x,y,z,n,h1,h2){
3  __CPROVER_assume(h1!=h2);
4  //h1,h2 are high
5  i1=1;i2=1;v1[1]=1;v2[1]=1;
6  while(i1+1<n&&i2+1<n)
7  { v1[i1] = h1+i1;
8    v2[i2] = h2+i2;
9    i1++; i2++;}
10 if(h1){ b1=x-y;
11   c1=x+y;a1=b1-c1;}
12 else{b1=x-y;a1=b1+z;}
13 if(h2){ b2=x-y;
14   c2=x+y;a2=b2-c2;}
15 else{b2=x-y;a2=b2+z;}
16 out1 = v1[i1-1] + a1;
17 out2 = v2[i2-1] + a2;
18 assert(a1==a2);
19 assert(b1==b2);
20 assert(c1==c2);
21 assert(i1==i2);
22 assert(v1[i1-1]==v2[
       i2-1]);
23 assert(out1==out2); }
```

Fig. 3: CBMC input for the generated optimized program P_1

1) Data types: For each internal temporary variable x, we create two copies, such as $x1$ and $x2$. The members declared inside a structure or a union can be responsible for an IL. Thus, we generate two copies of the structure variable or union variable instead of making copies of the members declared in a structure or union. When one or more elements of the array have an information flow from any H_in, it is considered a single leak of the array, as the rest of the elements of the array can also be accessible through the leaky element address. Thus, we generate two copies of the arrays, just like variables. A pointer variable may store the address of a leaky variable. So, we generate two copies of the pointer variable of the same type.

2) Control Structures: The control structure in a program is responsible for implicit information flows. We generate two copies of the control block in a program. For the control block presented in Fig. 2(b), the generated control block is presented in Fig. 3 Lines 10 to 15. When we assume the two copies of H_in h, i.e., $h1$ and $h2$ have different values, i.e., $h1! = h2$ (Sec. IV), it is possible that the true branch of the first copy of the control block and the false branch of the other control block or vice versa are enabled during the execution of the program. Thus, in Fig. 3, the program execution generates different values for the copies of variables a and c, i.e., the assertions for $a1 == a2$ and $c1 == c2$ fail due to their different symbolic values. However, the assertion for variable b passes due to their same symbolic values in parallel paths. Thus, only a and c are considered leaky variables due to implicit flows. This is how our proposed approach precisely analyzes implicit flows in a program and overcomes the problem of over-tainting, as discussed in Sec. II.

3) Loops: To find the fixed point of the loop leak, we generate two copies of the loop and merge them. Loop merging is done for parallel execution of copies of the loop. For the loop in Fig. 2(b), the generated loop is presented in Fig. 3 Lines 6 to 9 where the two copies are merged.

Note that CBMC cannot handle unbounded loops. The option $--unwind\ k$ is used to fix the bound k for the loops in the program while running CBMC. We perform a pre-analysis to find the minimum number of iterations a loop needs to iterate to find the fixed point of the leak in the loop using the approach of [8]. A

Authorized licensed use limited to: Indian Institute of Technology (ISM) Dhanbad. Downloaded on August 04,2026 at 12:14:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Embedded Systems Letters. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/LES.2026.3677498

loop may have a number of parallel paths inside it due to control blocks. Consider a loop with two parallel paths, $P1$ and $P2$. In each iteration of the loop, it follows either $P1$ or $P2$. Then, our goal is to check all possible path sequences to get the fixed point of the leak in the loop, like $P1$, $P2$, $P1P1$, $P1P2$, $P2P1$, $P2P2$, $P1P1P1$, etc. It is possible that in each iteration, a new variable is introduced as a leaky variable. Thus, the depth or maximum number of iterations for a loop would be the order of V in the worst case. We use this fixed point value as the unwind value k in CBMC for this loop. This will ensure that CBMC unrolls the loop enough times to identify all possible leaks inside the loop. The assertion for copies of the array v in Fig. 3 fails due to the information flow from H_in h to the array v in the loop. It may be noted that most cryptographic benchmarks use bounded loops.

```
1  void main(h)
2  { int v; v=f(h);}
```
(a)

```
1  int f(x)
2  {int y;y=x+5;return y;}
```
(b)

```
1  void main(h1, h2)
2  {assume(h1!=h2);
3   int v1,v2;v1=f(h1);
4   v2=f(h2);f'(h1,h2);
5   assert(v1==v2); }
```
(c)

```
1  int f'(x1, x2)
2  {int y1,y2;
3   y1=x1+5; y2=x2+5;
4   assert(y1==y2);
5   assert(x1==x2); }
```
(d)

Fig. 4: Handling Function (a) Original main() function; (b) Original function definition f(); (c) Generated main() function; (d) Generated function definition f′()

4) Functions: For a function, information flow may occur through the local variables declared inside a function, and/or through the function return value. To capture this, we generate two copies of each function definition f() in a program, except the main function, in the following way. The first copy f(), uses the original definition of the function as it is (with the original return statement, if any). This copy is used to track the information flow from the function parameters to the value returned from the function. In the generated code, the first copy f() is called twice at the place of the original function call with the two copies of the argument variables. Consider the program with a function call in Fig. 4(a) and Fig. 4(b). In the generated code, there are two function calls with the copies $h1$ and $h2$ as parameters in Fig. 4(c), at the place of the original function call f(h) in Fig. 4(a). In the example, the leak due to the variable v of the $main()$ can now be tracked through the assert statement inserted in the generated $main()$ for the copies of the variable v as $v1$ and $v2$. In the second copy f′(), the arguments and the local variables declared in the original definition of f() are duplicated. This copy is used to identify the IL through the internal variables of the function; thus, it has no return value. For example in Fig. 4(c), function f′() is called after the two function calls of f(). Here, we add the assertions for each local variable and each function parameter at the end of the function definition f′(). For example, y is leaking in Fig. 4(b), so the assertion $y1 == y2$ will fail in this case.

V. Leak Parameters and Relative Security

In [8], the authors proposed three quantification parameters for the overall IL in a program: (i) Total leaky high inputs: when there is more than one H_in in a program, the leak of each H_in is checked separately. (ii) Unique leaky variables: the total number of unique leaky variables measured is with respect to any H_in. Here, a variable leaking more than one H_in is considered a single leak. (iii) Total leaky variables with respect to unique high input: a variable leaking two H_in is considered as two leaks.

All three quantification parameters can be obtained in our CBMC-based approach by controlling just the assumptions on the high inputs. Let us consider a program with two sensitive inputs $h1, h2 \in H_in$. For the first parameter, to check the leakage of $h1$, the two added assumptions are $h11 \neq h12$ and $h21 == h22$. Similarly, the leak of $h2$ is verified with the assumptions $h11 == h12$ and $h21 \neq h22$. For the second parameter, to count the unique leaky variables, the assumptions should be $h11 \neq h12$ and $h21 \neq h22$. Now, for the third parameter, to count the total leaky variables with respect to $h1$, the assumptions are the same as for the first parameter. With this setup, we verify whether any of the assertions failed. A sensitive input $h1 \in H_{in}$ is leaky if any of the added assertions fail for any variable in the program. The total number of failed assertions is the total number of leaky variables in the program.

A. Verifying Relative Security (RS)

An optimized program is said to be relatively secure to the source program if the number of variables leaking a specific $h \in H_in$ in the optimized program is not more than the number of variables leaking the same h in the source program. To verify the RS, we find the third quantification parameter, i.e., the total leaky variables with respect to the unique $h \in H_in$ for both the source and optimized programs. If for each H_in, the total number of leaky variables in the optimized program is less than or equal to that of the source program, then the optimized program is relatively secure to the source program. For example, in Fig. 2, there is only one sensitive input h which is leaked through a, u, and v in P_0 and through a, c, and v in P_1. Hence, this optimized program P_1 is relatively secure to the source program P_0.

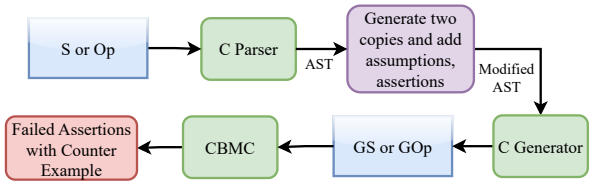


Fig. 5: Implementation of MQIL for quantification of IL

VI. Experimental Results

We have considered four well-known cryptographic algorithms, namely AES, DES, SHA and RSA in our experimentation. For each test case, LLVM is used to generate an optimized LLVM IR (with -O3) from source C (S). Then, the tool `llvm2c` [10] is used to translate the optimized LLVM IR into a semantically equivalent C program (Op). It is worth noting that `llvm2c` performs a direct, instruction-level translation of the IR into its corresponding C constructs without applying additional optimizations, unlike the LLVM optimization pipeline. Therefore, the translated C code preserves the data and control dependencies present in the optimized IR. The functional correctness of both the source and optimized C is checked using a simulation-based method. Our proposed approach MQIL is implemented in Python. We use the pycparser to obtain the abstract syntax tree (AST) from S and Op. The creation of miter program for quantification of IL (ref.

Authorized licensed use limited to: Indian Institute of Technology (ISM) Dhanbad. Downloaded on August 04,2026 at 12:14:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Embedded Systems Letters. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/LES.2026.3677498

Table I: Characteristics of Source (S) and Generated Source (GS)

Bench	#LOC		#Var		#Loop		#Func.		ET (s)
	S	GS	S	GS	S	GS	S	GS	
AES	358	1182	23	91	10	35	12	44	0.42
DES	247	782	44	150	20	58	7	26	0.17
SHA	171	565	30	97	12	31	13	42	0.25
RSA	66	136	13	45	3	6	4	15	0.04
Avg.	**2.95×**		**3.51×**		**2.74×**		**3.59×**		

Fig. 3) is implemented at the AST level. The miter C codes, i.e., GS and GOp from S and Op, respectively, are generated by the C generator in the pycparser library from the modified AST. We used CBMC 5.10 to quantify the IL and verify the RS between the source and optimized C code. Fig. 5 shows the tool flow of our implementation. In our experiments, we used an Intel Xeon(R) CPU E5-2620 v4 2.10GHz, 64GB of RAM, running Ubuntu 18.04.3 LTS.

Table II: Characteristics of Optimized (Op) and Generated Optimized (GOp)

Bench	#LOC		#Var		#Loop		#FuncCall		ET (s)
	Op	GOp	Op	GOp	Op	GOp	Op	GOp	
AES	998	3496	287	1620	10	22	5	19	1.12
DES	404	1425	149	467	14	41	4	14	0.46
SHA	479	1751	111	339	12	10	27	94	1.49
RSA	105	490	30	99	0	0	1	3	0.07
Avg.	**3.83×**		**3.78×**		**1.49×**		**3.44×**		

We have presented the experimental results in Tabs. I to III for four cryptographic benchmarks, AES, DES, SHA, and RSA. We considered the encryption mode for AES and DES, and the secret key is considered as H_in. The digest info is taken as H_in for SHA to generate the hash value. In RSA, the private key is considered the H_in in our experiment. The characteristic of the source code (S) and the generated source code (GS) (i.e., the version of S that will be input to CBMC) for each benchmark in terms of lines of code (#LOC), variables (#var), loops (#loop), and function calls (#funccall) is presented in Tab. I. The #LOC, #var, #loop, and #funccall are increased by 2.95, 3.51, 2.74, and 3.59 times, respectively, due to the two copies of the program. The #funccall shown here excludes the assume and assertion functions in the GS. The values for the #funccall vary in the range of three to four times in GS than in S. Our tool takes less than a second to generate GS from S, as shown in the last column of Tab. I. Similar results are presented for optimized (Op) and generated optimized code (GOp) in Tab. II. #LOC, #variable, #loop, and #funccall are increased by 3.83, 3.78, 1.49, and 3.44 times, respectively. The loops are unrolled in the Opt of RSA, thus #loop in Op and GOp for RSA is 0. We observed that LLVM optimizes some of the function calls during the Opt, thus #funccall is reduced in the Op than the S for AES, DES, and RSA, i.e., from 12 to 5, 7 to 4, and 4 to 1, respectively. Interestingly, we also observed that LLVM introduces new functions in its Opt phase; thus, #funccall is increased to 27 in the Op from 13 in S for SHA.

We have presented the performance measures for the benchmarks in Tab. III. For each benchmark, the number of inputs (#in), and high inputs (#high) is shown in columns 2 and 3, respectively. Column 4 shows the number of assumptions (#assume) in the code. Columns 5 and 6 show the total number of assertions (#assert) in the generated source (GS) and optimized code (GOp),

Table III: Performance Measures for Benchmarks in CBMC

Bench	#In	#High	Assume	Assert.		Leak		RS?	ET (s)	
				GS	GOp	GS	GOp		GS	GOp
AES	2	1	2	28	279	8	157	No	54.29	24.72
DES	3	2	5	57	150	14	66	No	21.55	14.55
SHA	2	1	2	32	142	3	11	No	0.31	3.09
RSA	5	1	5	5	21	2	1	Yes	0.01	5.58

respectively. Inputs and assumptions are the same for both GS and GOp. As LLVM IR is in SSA form, it introduces a large number of new variables during the Opt. Thus, the #assert is increased drastically for GOp than GS. The #leak in the program is the total number of assertions failed in CBMC. The total number of leaks is presented in columns 7 and 8 for both GS and GOp, respectively. The relative security (RS) between GOp and GS is presented in column 9. The results show that the optimized program is not relatively secure compared to the source program for all the cryptographic benchmarks in our experiment, except for RSA. This shows that LLVM introduces new security vulnerabilities in its optimization phase. The average execution time for CBMC show that MQIL takes less than a minute to quantify the leak for large benchmarks like AES and DES. This signifies our proposed method is scalable enough to handle even larger benchmarks. It may be noted that the method in [8], [9] cannot handle any of these benchmarks. Although we instantiate the framework for C, the underlying structure applies to other languages as well. However, languages with fundamentally different memory and execution models require redefinition of the semantic and observation components, which is beyond the scope of this work.

VII. Conclusion and Future Work

We propose a generic property checking based method to quantify information leakage and demonstrate it on C programs using CBMC. The proposed approach measures both the explicit and the implicit leaks, in addition to the fixed point of leak in a loop. The approach is further extended to verify the relative security between a source program and its optimized version in a compiler. The experimental results show that the proposed approach is scalable. In the future, we plan to further explore the possibility of post-fixing the ILs introduced by the LLVM compiler framework in its optimization phase.

References

[1] A. Pnueli, M. Siegel, and E. Singerman, "Translation validation," in *TACAS*. Springer-Verlag, 1998, p. 151–166.

[2] F. Besson, A. Dang, and T. Jensen, "Information-Flow Preservation in Compiler Optimisations," in *CSF*, 2019, pp. 1–13.

[3] C. Deng and K. S. Namjoshi, "Securing a compiler transformation," in *Static Analysis*, X. Rival, Ed., 2016, pp. 170–188.

[4] V. D'Silva *et al.*, "The correctness-security gap in compiler optimization," in *IEEE Security and Privacy Workshops*, 2015, pp. 73–87.

[5] R. Sadhukhan, S. Saha, and D. Mukhopadhyay, "Shortest path to secured hardware: Domain oriented masking with high-level-synthesis," in *ACM/IEEE DAC*. IEEE, 2021, pp. 223–228.

[6] A. Geimer and C. Maurice, "Fun with flags: How compilers break and fix constant-time code," *arXiv preprint arXiv:2507.06112*, 2025.

[7] D. E. Bell and L. J. LaPadula, "Secure computer systems : Mathematical foundations," 1973.

[8] P. Panigrahi, A. Paul, and C. Karfa, "Quantifying information leakage for security verification of compiler optimizations," *IEEE TCAD*, vol. 41, no. 11, pp. 4385–4396, 2022.

[9] P. Panigrahi and C. Karfa, "Translation validation of information leakage of compiler optimizations," *IEEE TCAD*, vol. 42, no. 11, pp. 3585–3598, 2023.

[10] Translation of LLVM bitcode to C, https://github.com/staticafi/llvm2c, accessed May 05 2024.

Authorized licensed use limited to: Indian Institute of Technology (ISM) Dhanbad. Downloaded on August 04,2026 at 12:14:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.